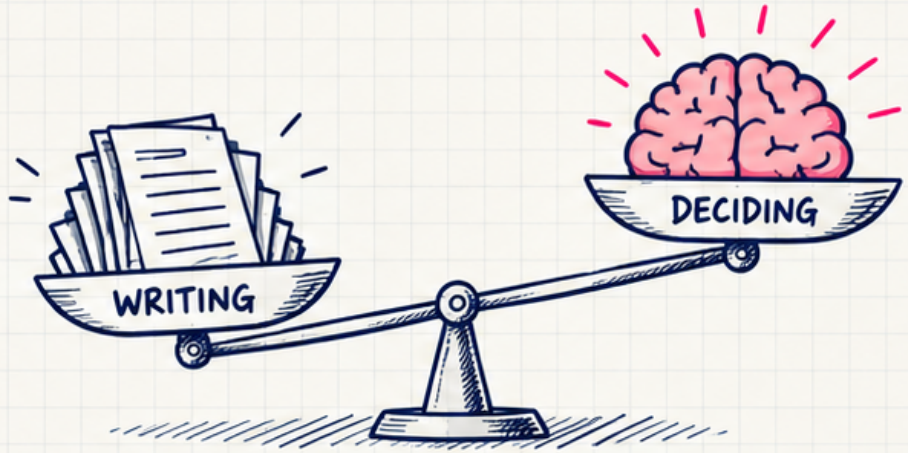
DEVELOPMENT WORKFLOW

The bottleneck moved from writing to deciding

Human attention is the scarce resource.



80%
562 of 701 commits
Agent co-author • since July 2026
Measured 2026-10-12

0
GitHub reviews in 100 merged PRs
Baseline • 2026-09-30

32%
63 of 200 PRs followed by a fix
Within 72 hours • baseline 2026-09-30

1
Human owner
Solo approval gap

Before: equal attention

After: attention by risk



Illustrative, not measured proportions.

Decide how much to read, and which lines.

Follow-up-fix rate has not been re-measured after triage.

DEVELOPMENT WORKFLOW

One change, start to finish

Six stages. Two human decision points.

1 Decide — HUMAN



Intent → spec → plan

2 Build — AGENT



Written contract → branch → PR

3 Triage — MACHINE



Skip / skim / read + reasons

4 Check — MACHINE



22 validation jobs

5 Merge & apply — HUMAN + MACHINE



Human decides; machine applies

6 Observe & record — MACHINE + AGENT



Ledger, docs, memory, next PRs

Infrastructure changes apply before merge.

Lessons go where the next change reads them.

DEVELOPMENT WORKFLOW

MACHINE

Triage guides attention

Labels and explanations. No merge veto.

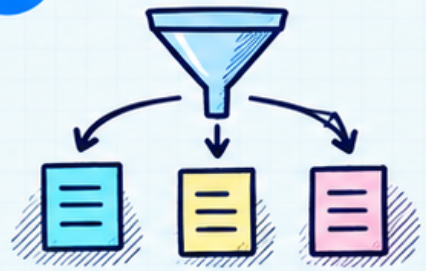
1 Rules



Paths, kinds and signals

Undecided

2 Small classifier



Skip / skim / read probabilities

Uncertain

3 Frontier model



Only ambiguous cases

review:skip | review:skim | review:read

Why • files and hunks • next commands

The label allocates attention. The human still decides.

- Policy from base commit
- CI changes → read
- Failure → read
- Triage never blocks

MACHINE

The wall of checks

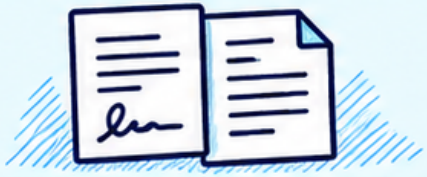
22 workflow jobs. Four kinds of validation.

Some jobs are conditional

Lint & schema
Syntax, manifests, infrastructure



Contracts
Docs, references, identity, tools



Generated-file drift
Regenerate and compare



Real-system drills
Boot a cluster. Test policies. Restore backups.



Static tools missed policy behavior twice.

Turn known failure modes into executable checks.

In the reference: schema tools + validators + generators + cluster drills

⚠ Checks reduce risk; they do not catch everything.

DEVELOPMENT WORKFLOW

Merge is the deployment

HUMAN + MACHINE

Apply order is part of change control.

Infrastructure changes: while the PR is open



Before merge

Cluster changes: after merge



Prune + self-heal
Removed resources pruned; drift corrected.

Solo-owner gap
Required approval uses an admin override.

Infrastructure first.
Merge next. Reconcile the cluster.

In the reference: infrastructure runner • GitOps controller • admission policy

DEVELOPMENT WORKFLOW

RECORDED • 2026-10-12

A triage comment caught its own mistake

A version string in a document triggered the rule.



COMMENT

review:read

Why: always_read.signals k3s_version_change

What to look at:

No k3s version change detected — The signal appears to be a false positive; this PR adds documentation and presentation files only, with no changes to infrastructure code, manifests, or Kubernetes configuration

Signals: decided_by: rules • size: large (1082 lines)

Trimmed recorded comment; full excerpt in speaker notes.

1 **Rule fired**



Document text matched a version signal.

2 **Explainer noticed**



The comment identified the false positive.

3 **Human decides**



The read label remained advisory.



The explanation corrected the interpretation, not the label.

DEVELOPMENT WORKFLOW

Reuse the mechanisms. Decide the ownership.

What maps to a company — and what does not yet.

01 Maps directly

- [x] Risk policy in Git
- [x] Advisory triage
- [x] Docs contracts + provenance
- [x] Generated-file checks
- [x] Real-system drills
- [x] Agent commit provenance



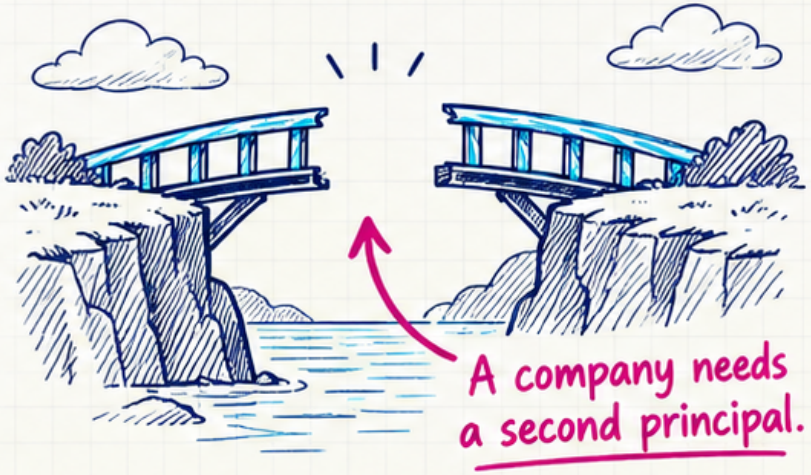
02 Needs a decision

- [ ] Who owns risk thresholds?
- [ ] Who reads a read PR?
- [ ] Which checks must block?
- [ ] May an agent merge?



03 Gaps to close

- [ ] Solo-owner admin override
- [ ] Calibration from one repository
- [ ] Reviewer tools are not a service



Start here

1. One repository →
2. Rules only →
3. One month of labels →
4. Calibrate →
5. Add classifier

Decide the operating model before scaling agent-written changes.